A

ALBAHIE AUCTION HOUSE

A DIVISION OF ABU ISSA GROUP

# Prototype Delivery Report

*The working system, the technology delivered, and how it maps to the original proposal.*

| DOCUMENT | STATUS | PREPARED BY | DATE |
| --- | --- | --- | --- |
| Build Report · v1.0 | Working prototype | AlBahie team & Mo | July 2026 |

CONTENTS

# What's inside



**How to read this document**

The May proposal was a *brainstorm* — a map of the whole operation and a candidate architecture. This report is its counterpart: a plain account of what has actually been built and is running, what technology it runs on, and where the delivered stack deliberately differs from the proposed one. Every claim here describes code that exists in the prototype today. Capabilities that are not yet built are labelled PHASE 2 rather than described as done.

SECTION 00

# § Executive summary

The proposal described an auction house run on a single source of truth: one item, one barcode, carried from the receiving counter through appraisal, agreement, catalogue, live sale, invoicing, and settlement. That system now exists as a working prototype. A staff member can log in, receive an item, run it through a manager decision, route it to a sale, catalogue it as a lot, run a live auction with real-time bidding and video, and issue a Stripe invoice — end to end, in one application.

| 9 | 17 | 11 | ~16k |
|---|---|---|---|
| LIVE MODULES | DATABASE TABLES | MIGRATIONS | LINES OF CODE |

The most important decision made during the build was **architectural, not functional**. The proposal specified a self-hosted, multi-service stack — a separate API server, a separate identity server (Keycloak), separate object storage (MinIO), a separate workflow engine (n8n), and a separate reporting tool (Metabase), all to be operated on rented infrastructure. For a prototype whose job is to prove the operating model quickly and be easy to demonstrate, that is a great deal of machinery to stand up and keep running.

We therefore **consolidated seven proposed services into one unified platform**: a single Next.js application backed by Supabase (managed PostgreSQL, authentication, file storage, and row-level security in one place). The vision did not change. The delivery vehicle did. Every capability that was deferred — handwriting OCR, the Invaluable bridge, predictive intake scoring — was deferred exactly where the proposal itself placed it: in Phase 2, because those capabilities depend on production data the prototype is designed to start collecting.

**The one-line version**

We kept the proposal's operating model and its security posture, and swapped a seven-service self-hosted architecture for one managed, single-runtime platform — which is why there is working software to show, not just a plan.

SECTION 01

# 1 What was built

The prototype is an invite-only, role-aware web application. There is no public sign-up: administrators provision staff accounts, exactly as the proposal required. Once signed in, a user sees a launchpad of modules gated by their role and a per-module permission matrix. Nine modules are working; one (Logistics) is scaffolded as the next to build.

## The item lifecycle, working end to end

| Stage (proposal §3) | In the prototype |
|---|---|
| Intake | Receive an item, capture photos, auto-generate the consignment (CN-) and delivery-note (DN-) references, print a QR + barcode label, and produce a printable delivery note. |
| Appraisal (14-day window) | A manager queue surfaces items by urgency. Accept, reject, or extend in weekly increments — every pending item always carries a forced future date. |
| Acceptance routing | On acceptance, route to auction, direct private sale, or inventory, and generate the matching consignment agreement with editable commercial fields. |
| Catalogue | Create a sale, pull accepted inventory in as ordered lots, and set the increment ladder and soft-close. |
| Live auction | Run the sale live: real-time bidding over WebSocket, absentee proxy bids, soft-close extension, OBS video to remote viewers, and the fall of the hammer — all written to an append-only audit trail. |
| Invoicing | Generate a buyer invoice (hammer + buyer's premium), issue it through Stripe as a hosted, payable invoice, email the link, and reconcile settlement. |
| History | Every step writes to a unified per-item activity log, so any item's full history is visible from any module. |

**What "working prototype" means here**

Every stage above is exercised by real code against a real database with row-level security. It is a prototype — not yet hardened for production traffic, and several proposal capabilities are Phase 2 — but it is a genuine end-to-end system, not screens over mock data.

SECTION 02

# 2 Implemented features, module by module

### Consignments LIVE

Intake form with photos, consignor picker, auto references, printable delivery note, QR/barcode labels, KPI header, and appraisal-deadline reminders routed to the responsible manager.

### Appraisal LIVE

Manager decision queue (overdue / due soon / upcoming), accept–reject–extend with a due-date preview, and a full decision history. No item sits silently.

### Inventory LIVE

Cross-cutting view of all property: search, status and location filters, table/card toggle, barcode scanner, location moves, and a merged activity timeline.

### Catalogue & Lots LIVE

Create sales, pull accepted inventory into a catalogue, order lots, and manage the increment ladder and soft-close per sale.

### Clients & KYC LIVE

Buyer/seller directory with KYC standing (verified, pending, rejected, expired), document uploads, paddle registrations, and per-client history.

### Live Auction LIVE

Deterministic bidding engine (increment ladder, absentee proxy bidding, soft-close) under unit test; authoritative WebSocket server, one room per lot; OBS→LiveKit video; bidder view, auctioneer console, and saleroom presentation screen; append-only bid audit.

### Invoicing & Payments LIVE

Invoices from hammer results, issued through Stripe as hosted invoices, emailed to the buyer, with settlement sync and a server-generated PDF.

### Reports LIVE

Sale performance — hammer, premium, settled totals and category breakdowns — with date/status filters and CSV export.

### Administration LIVE

Admin-only: user management, custom roles with a per-module Create/Read/Update/Delete matrix, and sale-event administration.

### Logistics & Shipping PHASE 2

Registered in the launchpad as the next module: storage, collection, and shipment of property.

## Roles & permissions

The proposal defined eight stakeholder profiles with hard, server-side permission boundaries. The prototype implements this as a flexible model rather than eight hard-coded roles: a base `admin` / `staff` distinction plus *custom roles*, each carrying a per-module CRUD matrix stored in the database. The eight proposed profiles (Administrator, Department Manager, Assistant Manager, Auctioneer, Accountant, Operations Staff, and the two buyer types) are all expressible in this model, and permissions are enforced in the server actions, not merely hidden in the interface — precisely the boundary the proposal insisted on.

## Audit & the single source of truth

Commercially significant actions — intake, edits, location moves, and appraisal decisions — are written to a per-item activity log with actor and timestamp. This is the operational spine the proposal described: one barcode, one history, inspectable from every module.

SECTION 03

# 3 The technology delivered

The delivered stack is deliberately lean and single-runtime: everything is TypeScript, and there is one application and one managed data platform rather than a fleet of self-hosted services.

| Layer | Technology | Role |
|---|---|---|
| Application & UI | Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4 | One codebase for the staff interface and the server logic. Server Components read data; Server Actions handle every mutation. Matches the proposal's Next.js + Tailwind client layer. |
| Data, auth & storage | Supabase — managed PostgreSQL 15, Auth, Storage | The database, authentication (invite-only email + password), and file storage for photos and documents — with row-level security on every table. |
| Live auction | Node WebSocket server + LiveKit | An authoritative bidding server (one room per lot, append-only audit) and low-latency OBS video to remote bidders and the saleroom screen. |
| Payments | Stripe Invoicing | Hosted, payable buyer invoices; card data never touches our database. Kept exactly as proposed. |
| Documents & labels | pdf-lib · qrcode · jsbarcode | Server-generated invoice PDFs and scannable QR + barcode item labels, all in the Node runtime. |
| Email | nodemailer (SMTP) | Transactional email (invoice links); provider-agnostic and swappable. |
| Icons & design system | lucide-react + a bespoke brand token layer | A single AlBahie design system — heritage teal and auction gold — expressed as Tailwind theme tokens. |

**Why single-runtime matters for a pilot**

Because the whole system is one TypeScript application over one managed database, it deploys in one step, has one place to secure, and has no service-to-service plumbing to debug. That is the difference between a prototype you can demonstrate next week and an infrastructure project you are still assembling.

SECTION 04

# 4 What we changed from the proposal, and why

The proposal's architecture was sound; it was optimised for a self-hosted, cost-minimised production system. The prototype is optimised for something different — proving the model fast with the fewest moving parts. Each change below trades operational machinery for delivery speed, without giving up the capability or the security the proposal called for.

| Concern | Proposed | Delivered | Why the change |
|---|---|---|---|
| Backend / API | FastAPI or NestJS as a separate tier | Next.js Server Actions & Route Handlers | One TypeScript codebase, type-safe from database to button. No separate API service to build, deploy, or keep in sync for a pilot. |
| Identity | Keycloak (self-hosted IAM) | Supabase Auth | Same invite-only, MFA-capable, role-based model without operating a Keycloak cluster. Roles live in `profiles` + `app_roles`, enforced by database row-level security. |
| Database | PostgreSQL, self-managed | Supabase (managed Postgres 15) | The *same* Postgres engine — pgvector still available for the proposal's similarity search — but managed, with auth and storage integrated. Removes the devops burden during the build. |
| Object storage | MinIO (self-hosted S3) | Supabase Storage | Identical S3 semantics for photos and signed documents, sharing the same auth and access rules. One fewer service to run. |
| PDF rendering | WeasyPrint + Jinja2 (Python) | pdf-lib (Node) | Keeps the stack single-runtime. No Python service in the critical path just to print an invoice. |
| Barcodes | python-barcode / Zint | qrcode + jsbarcode (Node) | Node-native, runs inside the app; produces the QR + barcode labels the intake process needs. |
| Workflow engine | n8n | In-app actions + scheduling | Prototype logic (reminders, deadlines) lives in the app. n8n can be reintroduced for notification routing later without touching the domain model. |

| | | | |
|---|---|---|---|
| Reporting / BI | Metabase | In-app Reports + CSV export | Native reporting the team can use immediately. Metabase can point at the same managed Postgres whenever richer self-serve dashboards are wanted. |
| Email / messaging | Resend / Mailgun / Twilio | nodemailer (SMTP) | Simple transactional email for the pilot; a drop-in swap to Resend or Twilio/WhatsApp later, as the proposal anticipated. |
| Live video | Invaluable live stream | OBS → LiveKit, native | An **addition beyond the proposal**: rather than depending on a third party to stream, the prototype runs its own saleroom video and real-time bidding. |
| Payments | Stripe (+ PayTabs option) | Stripe | Unchanged. The regional PSP option (PayTabs/Tap) remains open for later. |

**The thread running through every row**

We did not remove any capability — we relocated it. Where the proposal ran a dedicated service, the prototype uses a managed equivalent or an in-app implementation, chosen so the pilot could be built and demonstrated quickly. The proposal's own principle was "use the ecosystem before reinventing it." Consolidating onto a managed platform is that principle applied to our own infrastructure.

SECTION 05

# 5 What is deliberately deferred to Phase 2

The proposal was explicit that some capabilities depend on real production data and belong after the first auctions run. The prototype respects that phasing exactly. Nothing below is a gap in the plan — each item is sequenced where the proposal itself placed it, and the prototype is built to start collecting the data each one needs.

| Capability (proposal ref) | Today | Why Phase 2 |
|---|---|---|
| Handwriting OCR of the auctioneer's book (§5.2) | MANUAL ENTRY | Hammer prices are entered directly today. OCR needs a sample of the real book to design the extraction schema — a post-launch data task, as the proposal states. |
| Tiered premium + channel surcharge + VAT engine (§6) | FLAT PREMIUM | Invoices compute hammer + buyer's premium today. The tiered rates, per-channel surcharges, and per-jurisdiction VAT become a configuration table — the seam (premium and channel fields) already exists on the invoice. |
| 30% pre-authorization holds (§5.3, §9.2) | INVOICE SETTLEMENT | Stripe is integrated for hosted invoices; manual-capture pre-auth holds are the same Stripe primitive applied at registration, added once the bidder-portal flow is built. |
| Invaluable integration via certified partner (§10) | PHASE 2 | The channel/surcharge model is designed so a third-party platform is a configuration row, exactly as the proposal argued. The partner bridge is a Phase-2 workstream. |
| Predictive intake scoring (§13.4) | PHASE 2 | Needs historical sales to train against. The prototype is already recording the data — every sale, bid, and outcome — that the model will later learn from. |
| WhatsApp delivery, n8n automation, Metabase, e-signature | PHASE 2 | All additive, all swappable onto the existing platform without re-architecting. |
| Consignor payouts & settlement queue | PHASE 2 | Buyer-side invoicing is built; the seller-side payout that closes the financial loop is the natural next increment. |

**Why this is a strength, not a caveat**

A prototype that tried to build OCR and predictive scoring on day one would be guessing, because it would have no real data to build them against. By shipping the operating system first and instrumenting it to capture every sale, bid, and outcome, the prototype earns the data those Phase-2 capabilities require. In the proposal's words: the first auction produces data, the hundredth produces insight.

SECTION 06

# 6 The five operating principles, honoured

The proposal opened with five operating principles. Here is how the prototype stands against each.

| Principle (proposal, §Executive Summary) | Status | In the prototype |
|---|---|---|
| Every item carries one immutable barcode for its entire life. | MET | A reference/barcode is generated at intake and carried through every stage and every module. |
| Every commercial decision is logged and reversible with a full audit trail. | MET | Intake, edits, moves, and appraisal decisions write to a per-item activity log with actor and timestamp. |
| Every buyer is invoiced automatically. No manual paper trail post-auction. | SUBSTANTIALLY MET | Invoices are generated from hammer results and issued through Stripe with an emailed payment link; automatic-on-hammer generation is a small wiring step. |
| Every fee component is adjustable per partner, per item, per jurisdiction. | MODELLED | Premium and channel fields exist on the invoice; the full configurable fee table is the Phase-2 engine (§5 above). |
| Every report can be generated on demand, sliced by category, auction, or buyer. | MET | The Reports module produces sales, category, and buyer views on demand with CSV export. |

**Where the prototype stands**

Three of the five principles are fully met in working software; the other two are modelled and partially built, with a clear, small path to completion. For a prototype delivered to prove the operating model, that is the whole model standing up under its own weight.

SECTION 07

# 7 The interface — a professional tool

An auction house's internal system is judged in part on whether it looks like it belongs to a house that handles fine art. The interface was built to that standard, not to a generic template.

### A real brand identity

A single AlBahie design system — heritage teal with auction-gold accents, carried as design tokens through every screen, in both light and dark themes. A teal-to-gold hairline crowns every page; the Abu Issa Group lineage is present on the sign-in.

### A live operations dashboard

The home screen opens on the live state of the house: KPI tiles for items awaiting appraisal, items in custody, the next or currently-live auction, and unpaid outstanding — beside a **real-time activity feed** of every recorded change across the floor and a **"needs attention"** queue of items overdue or due for a decision. Everything deep-links into its module.

### Built for people who work fast

A module sidebar on every working screen, a Ctrl-K command palette for jumping to any module or action, keyboard-navigable throughout, and an accessible focus ring on every control.

### Considered detail

Consistent iconography, dense but legible data tables, printable delivery notes and agreements, empty and loading states, and a user-controlled light/dark theme with no flash on load.

The result reads as a purpose-built operating system for the auction house — the impression the proposal set out to create.

❖

*The proposal asked for the operating system of the auction house, with every step encoded, every decision logged, and every outcome measurable. This prototype is that system, standing and running — ready for the team to react to, refine, and take live.*